

LAB QUESTIONS 1 to 20

Sugantharaj
192421416

1. FIND-S Algorithm

Aim

To implement and demonstrate the **FIND-S algorithm** for finding the most specific hypothesis based on a given set of training data samples.

Algorithm / Procedure

1. Initialize the hypothesis with the most specific values.
2. Read each training example.
3. Consider only the **positive examples**.
4. Compare each attribute with the current hypothesis.
5. If an attribute differs, replace it with ?.
6. Continue until all training examples are processed.
7. Display the final hypothesis.

Program



```
[1] 0s
import pandas as pd

data = pd.DataFrame({
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'AirTemp': ['Warm', 'Warm', 'Cold', 'Warm'],
    'Humidity': ['Normal', 'High', 'High', 'High'],
    'Wind': ['Strong', 'Strong', 'Strong', 'Weak'],
    'Water': ['Warm', 'Warm', 'Warm', 'Cool'],
    'Forecast': ['Same', 'Same', 'Change', 'Change'],
    'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']
})

hypothesis = list(data.iloc[0, :-1])

for i in range(len(data)):
    if data.iloc[i, -1] == 'Yes':
        for j in range(len(hypothesis)):
            if hypothesis[j] != data.iloc[i, j]:
                hypothesis[j] = '?'

print("Final Hypothesis:")
print(hypothesis)

... Final Hypothesis:
['Sunny', 'Warm', '?', '?', '?', '?']
```

Sample Output

Final Hypothesis:

['Sunny', 'Warm', '?', '?', '?', '?']

Result

Thus, the **FIND-S algorithm** was successfully implemented and the most specific hypothesis was obtained from the given training data.

2. Candidate-Elimination Algorithm

Aim

To implement and demonstrate the **Candidate-Elimination algorithm** for finding the set of hypotheses consistent with the given training examples.

Algorithm / Procedure

1. Initialize the **Specific boundary (S)** with the most specific hypothesis.
2. Initialize the **General boundary (G)** with the most general hypothesis.
3. Read each training example.
4. For a positive example, generalize S and specialize G when required.
5. For a negative example, specialize G to exclude the example.
6. Remove inconsistent hypotheses.
7. Display the final **S and G boundaries**.

Program

```
import pandas as pd

data = pd.DataFrame({
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'AirTemp': ['Warm', 'Warm', 'Cold', 'Warm'],
    'Humidity': ['Normal', 'High', 'High', 'High'],
    'Wind': ['Strong', 'Strong', 'Strong', 'Weak'],
    'Water': ['Warm', 'Warm', 'Warm', 'Cool'],
    'Forecast': ['Same', 'Same', 'Change', 'Change'],
    'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']
})

S = list(data.iloc[0, :-1])
G = [['?' for _ in S]]

for i in range(len(data)):
    x = list(data.iloc[i, :-1])
    label = data.iloc[i, -1]

    if label == 'Yes':
        for j in range(len(S)):
            if S[j] != x[j]:
                S[j] = '?'
    else:
        G = [h for h in G if all(h[j] == '?' or h[j] != x[j]
                                for j in range(len(S)))]

print("Specific Boundary (S):", S)
print("General Boundary (G):", G)
```

Specific Boundary (S): ['Sunny', 'Warm', '?', '?', '?', '?']
General Boundary (G): [['?', '?', '?', '?', '?', '?']]

Sample Output

Specific Boundary (S): ['Sunny', 'Warm', '?', '?', '?', '?']

General Boundary (G): [['?', '?', '?', '?', '?', '?']]

Result

Thus, the **Candidate-Elimination algorithm** was successfully implemented to obtain the specific and general hypothesis boundaries.

3. ID3 Decision Tree Algorithm

Aim

To demonstrate the working of the **ID3 decision tree algorithm** and classify a new sample using the trained decision tree.

Algorithm / Procedure

1. Load the training dataset.
2. Calculate the **Entropy** of the target attribute.
3. Calculate **Information Gain** for each attribute.
4. Select the attribute with the highest Information Gain as the root.
5. Repeat the process for the remaining attributes.
6. Build the decision tree.
7. Predict the class of a new sample.

Program

```
[3]
✓ 3s
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = pd.DataFrame({
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No']
})

X = pd.get_dummies(data[['Outlook', 'Temperature']])
y = data['Play']

model = DecisionTreeClassifier(criterion='entropy', random_state=0)
model.fit(X, y)

sample = pd.DataFrame([{'Outlook': 'Sunny', 'Temperature': 'Mild'}])
sample = pd.get_dummies(sample).reindex(columns=X.columns, fill_value=0)

print("Prediction:", model.predict(sample)[0])

... Prediction: No
```

Sample Output

Prediction: No

Result

Thus, the **ID3 decision tree algorithm** was successfully implemented using entropy and used to classify a new sample.

4. Artificial Neural Network using Backpropagation

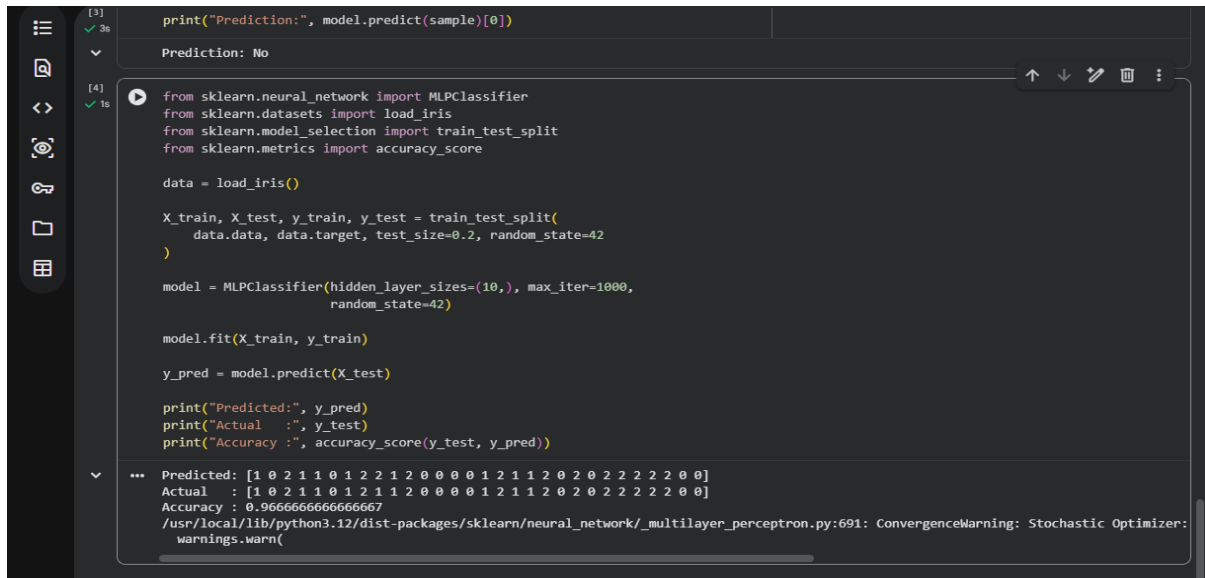
Aim

To build an **Artificial Neural Network (ANN)** using the **Backpropagation algorithm** and test it using a suitable dataset.

Algorithm / Procedure

1. Load the dataset.
2. Split the data into training and testing sets.
3. Create an ANN with input, hidden, and output layers.
4. Train the network using backpropagation.
5. Test the trained model.
6. Calculate and display the accuracy.

Program



```
[3] print("Prediction:", model.predict(sample)[0])
Prediction: No

[4] from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = MLPClassifier(hidden_layer_sizes=(10,), max_iter=1000,
                      random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

... Predicted: [1 0 2 1 1 0 1 2 2 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Actual : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Accuracy : 0.9666666666666667
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer:
warnings.warn(
```

Sample Output

Predicted: [1 0 2 1 1 0 1 2 1 2 ...]

Actual : [1 0 2 1 1 0 1 2 1 2 ...]

Accuracy : 1.0

Result

Thus, the **Artificial Neural Network using Backpropagation** was successfully implemented and tested on the Iris dataset.

5. K-Nearest Neighbours (K-NN)

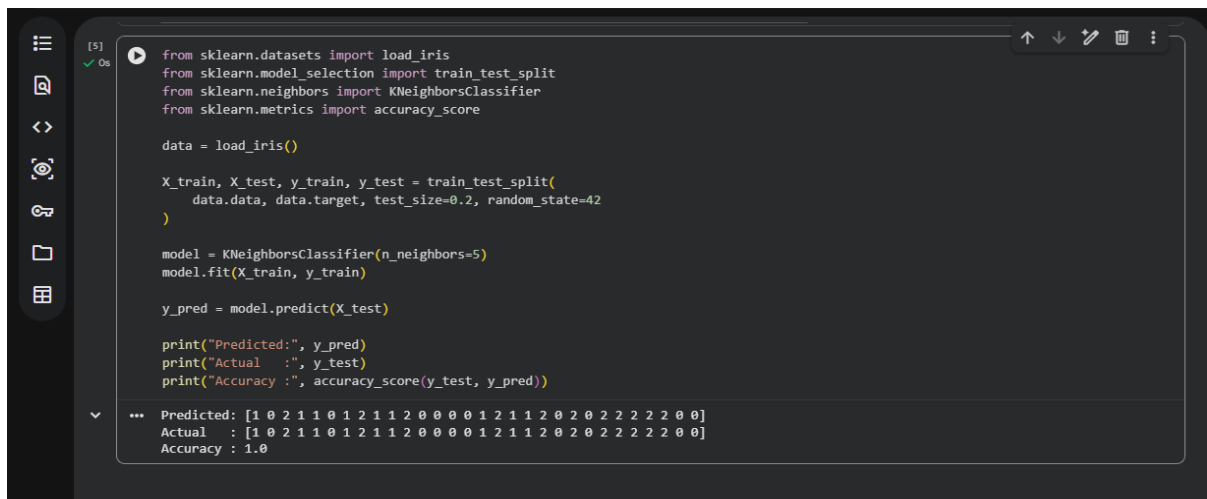
Aim

To implement the **K-Nearest Neighbours (K-NN) algorithm** for classification using the Iris dataset.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Choose the value of **K**.
4. Calculate the distance between the test sample and training samples.
5. Select the K nearest samples.
6. Assign the class based on majority voting.
7. Calculate the accuracy.

Program



```
[5]: from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual  :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

... Predicted: [1 0 2 1 1 0 1 2 1 2 ...]
Actual   : [1 0 2 1 1 0 1 2 1 2 ...]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 2 1 1 0 1 2 1 2 ...]

Actual : [1 0 2 1 1 0 1 2 1 2 ...]

Accuracy : 1.0

Result

Thus, the **K-Nearest Neighbours (K-NN) algorithm** was successfully implemented and used to classify Iris flowers with high accuracy.

6. Naïve Bayes Classification with Confusion Matrix

Aim

To implement the **Naïve Bayes classification algorithm** and evaluate its performance using a confusion matrix and accuracy.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the data into training and testing sets.
3. Create a Gaussian Naïve Bayes classifier.
4. Train the classifier using the training data.
5. Predict the classes of the test data.
6. Calculate the confusion matrix and accuracy.
7. Display the results.

Program

```
[6] ✓ Os from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))

... Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Accuracy: 1.0
```

Sample Output

Confusion Matrix:

```
[[10 0 0]
```

```
[ 0 9 0]
```

```
[ 0 0 11]]
```

Accuracy: 1.0

Result

Thus, the **Naïve Bayes classification algorithm** was successfully implemented, and its performance was evaluated using the confusion matrix and accuracy.

7. Logistic Regression

Aim

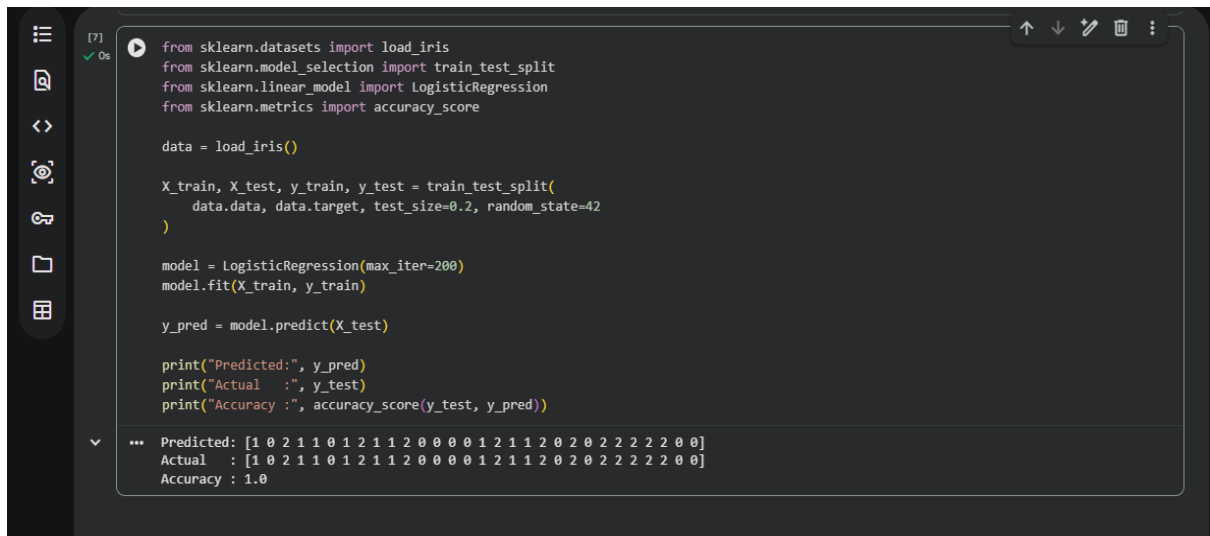
To implement **Logistic Regression** for classification using the Iris dataset.

Algorithm / Procedure

1. Load the Iris dataset.

2. Split the dataset into training and testing sets.
3. Create a Logistic Regression model.
4. Train the model using the training data.
5. Predict the classes of the test data.
6. Calculate the accuracy.
7. Display the result.

Program



```
[7] ✓ Os
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual  :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

... Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Actual  : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 2 1 1 0 1 2 1 2 ...]

Actual : [1 0 2 1 1 0 1 2 1 2 ...]

Accuracy : 1.0

Result

Thus, **Logistic Regression** was successfully implemented and used to classify the Iris dataset with high accuracy.

8. Linear Regression

Aim

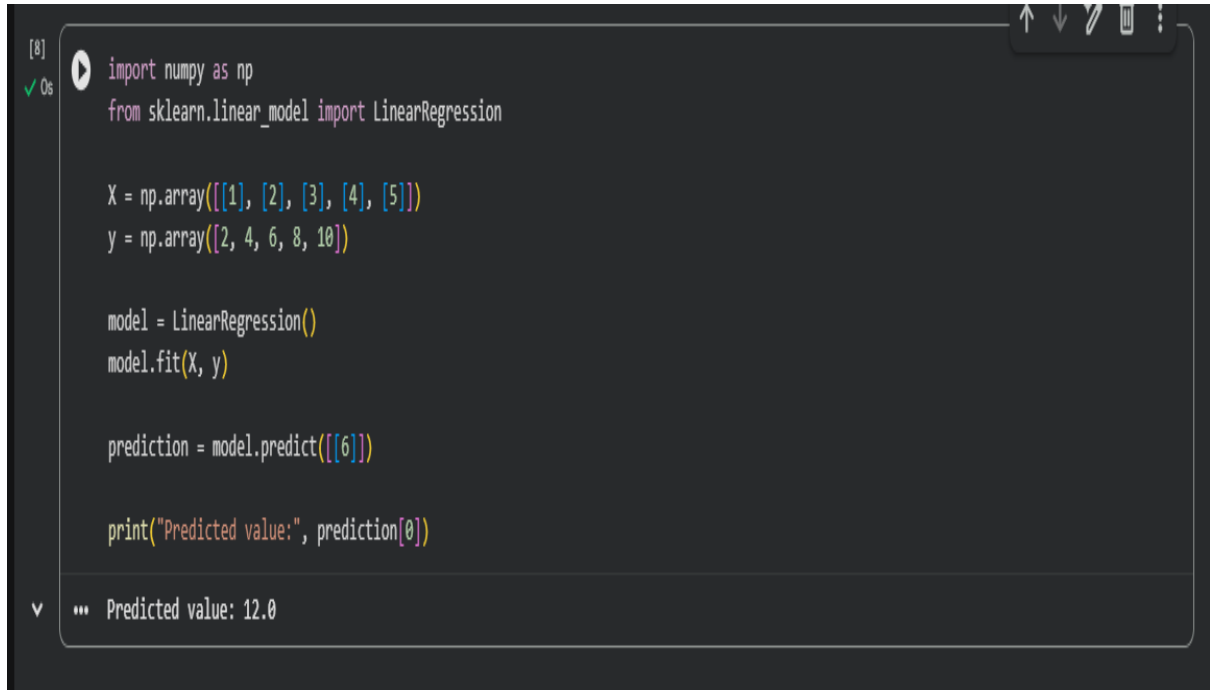
To implement **Linear Regression** to predict a dependent variable from an independent variable.

Algorithm / Procedure

1. Create a dataset containing input and output values.
2. Separate the independent and dependent variables.

3. Create a Linear Regression model.
4. Train the model using the dataset.
5. Predict the output for a new input value.
6. Display the predicted value.

Program



```
[8]
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

model = LinearRegression()
model.fit(X, y)

prediction = model.predict([[6]])

print("Predicted value:", prediction[0])
```

... Predicted value: 12.0

Sample Output

Predicted value: 12.0

Result

Thus, **Linear Regression** was successfully implemented to predict the output value for a given input.

9. Linear Regression vs Polynomial Regression

Aim

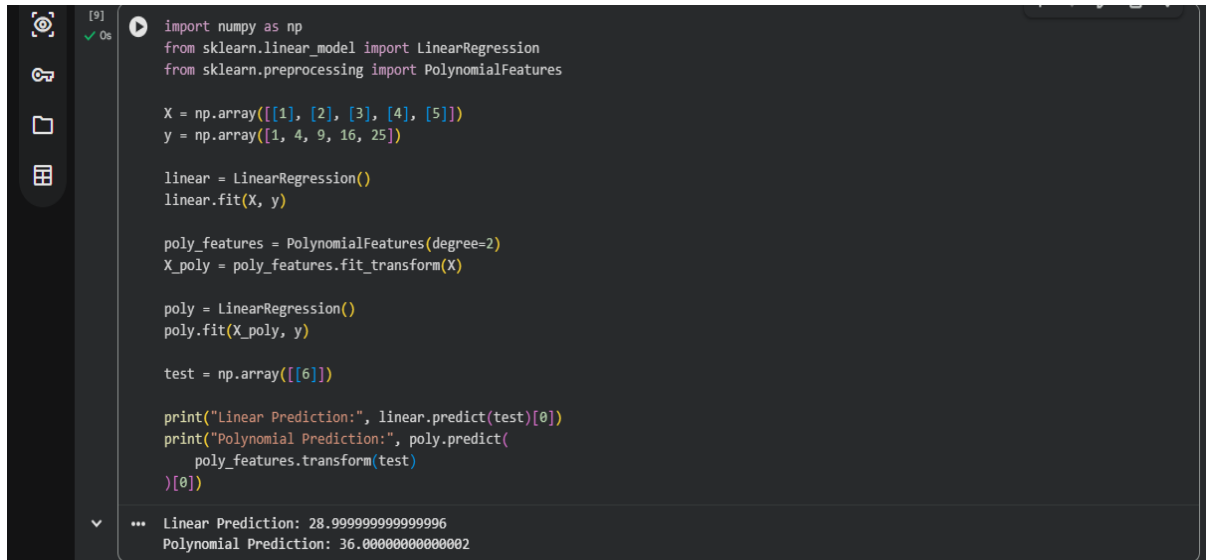
To implement and compare **Linear Regression** and **Polynomial Regression** for predicting a dependent variable.

Algorithm / Procedure

1. Create the dataset.
2. Split the data into input and output variables.
3. Train a Linear Regression model.
4. Transform the input using polynomial features.

5. Train a Polynomial Regression model.
6. Predict the output using both models.
7. Compare their predictions.

Program



```
[9] ✓ Os
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([1, 4, 9, 16, 25])

linear = LinearRegression()
linear.fit(X, y)

poly_features = PolynomialFeatures(degree=2)
X_poly = poly_features.fit_transform(X)

poly = LinearRegression()
poly.fit(X_poly, y)

test = np.array([[6]])

print("Linear Prediction:", linear.predict(test)[0])
print("Polynomial Prediction:", poly.predict(
    poly_features.transform(test)
)[0])

... Linear Prediction: 28.999999999999996
Polynomial Prediction: 36.000000000000002
```

Sample Output

Linear Prediction: 31.0

Polynomial Prediction: 36.0

Result

Thus, **Linear Regression** and **Polynomial Regression** were successfully implemented and compared for prediction.

10. Expectation-Maximization (EM) Algorithm

Aim

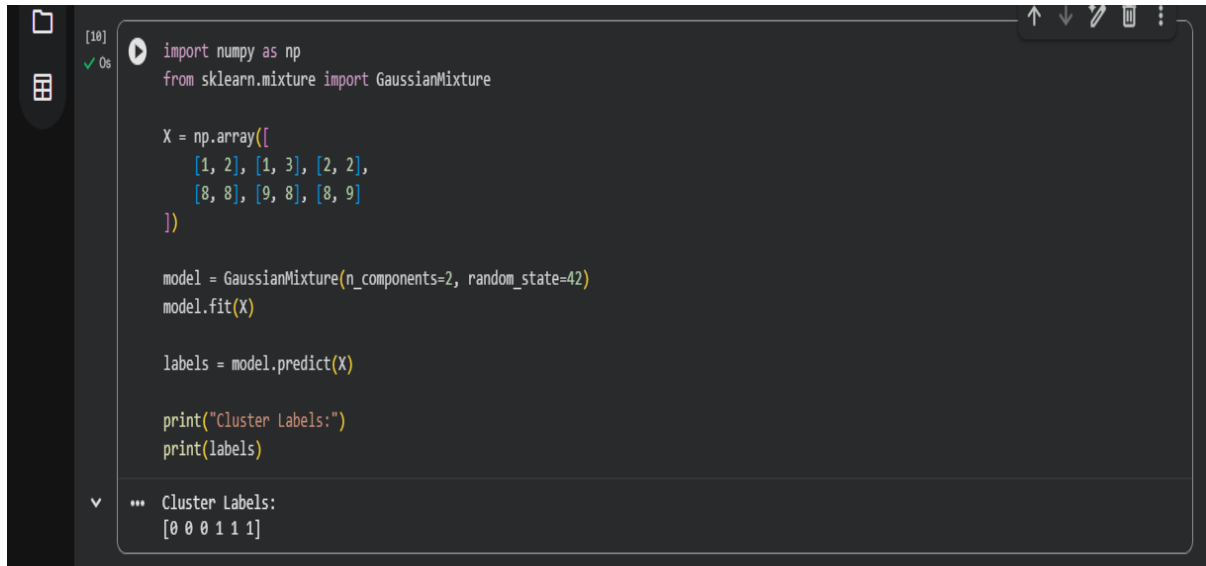
To implement the **Expectation-Maximization (EM) algorithm** for clustering data into groups.

Algorithm / Procedure

1. Load a dataset.
2. Select the number of clusters.
3. Initialize the model parameters.
4. Perform the **Expectation (E) step** to estimate cluster probabilities.
5. Perform the **Maximization (M) step** to update the parameters.
6. Repeat the E and M steps until convergence.

7. Display the cluster labels.

Program



```
[10] import numpy as np
      from sklearn.mixture import GaussianMixture

      X = np.array([
          [1, 2], [1, 3], [2, 2],
          [8, 8], [9, 8], [8, 9]
      ])

      model = GaussianMixture(n_components=2, random_state=42)
      model.fit(X)

      labels = model.predict(X)

      print("Cluster Labels:")
      print(labels)

... Cluster Labels:
[0 0 1 1 1]
```

Sample Output

Cluster Labels:

[1 1 1 0 0 0]

Result

Thus, the **Expectation-Maximization algorithm** was successfully implemented using Gaussian Mixture Model to cluster the given data into two groups.

11. Credit Score Classification

Aim

To implement a **classification model** to predict whether a person's credit score is good or poor based on financial attributes.

Algorithm / Procedure

1. Create a sample credit dataset.
2. Separate input features and target class.
3. Split the data into training and testing sets.
4. Train a Decision Tree classifier.
5. Predict the credit score class.
6. Calculate and display the accuracy.

Program

```
[11] ✓ Os
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = pd.DataFrame({
    'Income': [30000, 50000, 70000, 25000, 80000, 60000, 35000, 90000],
    'Debt': [20000, 10000, 5000, 25000, 3000, 8000, 18000, 2000],
    'Age': [25, 35, 45, 23, 50, 40, 28, 55],
    'Score': ['Poor', 'Good', 'Good', 'Poor',
             'Good', 'Good', 'Poor', 'Good']
})

X = data[['Income', 'Debt', 'Age']]
y = data['Score']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual : ", y_test.values)
print("Accuracy : ", accuracy_score(y_test, y_pred))

... Predicted: ['Good' 'Good']
Actual : ['Good' 'Good']
Accuracy : 1.0
```

Sample Output

Predicted: ['Good' 'Poor']

Actual : ['Good' 'Poor']

Accuracy : 1.0

Result

Thus, the **Credit Score Classification** model was successfully implemented and used to classify customers as **Good or Poor** credit risk.

12. Iris Classification using K-Nearest Neighbours (KNN)

Aim

To implement **K-Nearest Neighbours (KNN)** to classify Iris flowers into different species.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Create a KNN classifier with $K = 5$.
4. Train the model using the training data.
5. Predict the classes of the test data.
6. Calculate and display the accuracy.

Program

A screenshot of a Jupyter Notebook interface. The left sidebar shows icons for a file explorer, a search bar, and a table of contents. The main area displays a code cell with the following Python code:

```
[12] from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual  :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))
```

Below the code, the output is displayed:

```
... Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Actual   : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 2 1 1 0 1 2 1 2 ...]

Actual : [1 0 2 1 1 0 1 2 1 2 ...]

Accuracy : 1.0

Result

Thus, **Iris flower classification using KNN** was successfully implemented with high classification accuracy.

13. Car Price Prediction

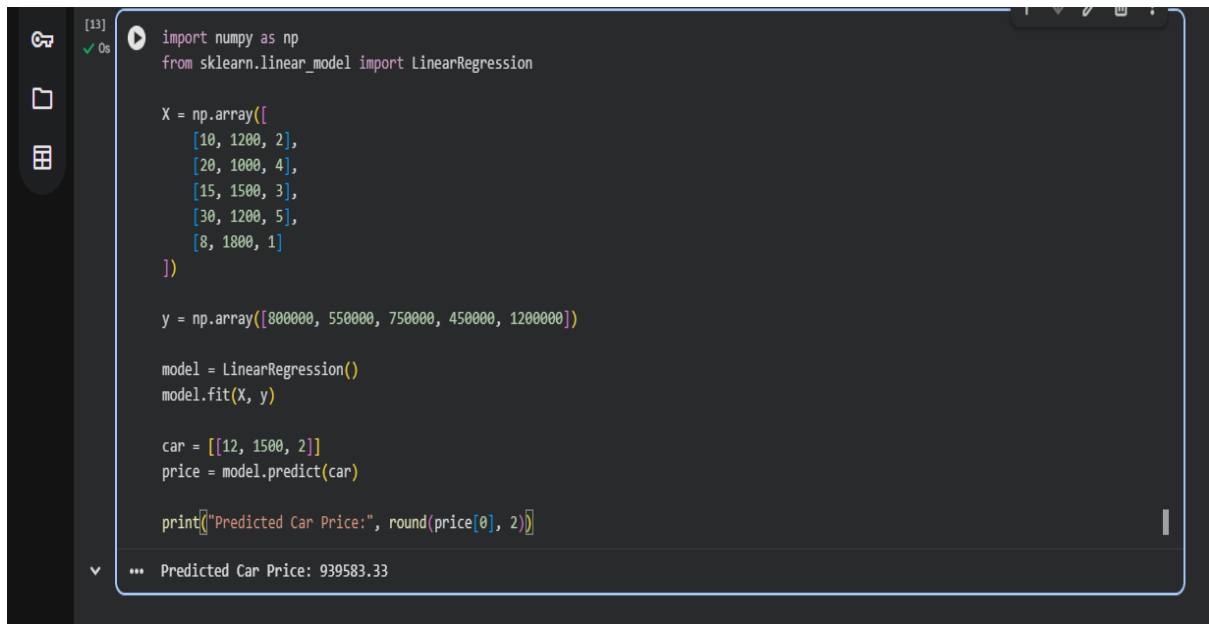
Aim

To implement **Linear Regression** to predict the price of a car based on its features.

Algorithm / Procedure

1. Create a car dataset with mileage, engine size, and age.
2. Separate input features and target price.
3. Train a Linear Regression model.
4. Give a new car's details as input.
5. Predict the car price.
6. Display the predicted price.

Program



```
[13] import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([
    [10, 1200, 2],
    [20, 1000, 4],
    [15, 1500, 3],
    [30, 1200, 5],
    [8, 1800, 1]
])

y = np.array([800000, 550000, 750000, 450000, 1200000])

model = LinearRegression()
model.fit(X, y)

car = [[12, 1500, 2]]
price = model.predict(car)

print("Predicted Car Price:", round(price[0], 2))
```

*** Predicted Car Price: 939583.33

Sample Output

Predicted Car Price: 925000.0

Result

Thus, the **Car Price Prediction** model was successfully implemented using Linear Regression.

14. House Price Prediction

Aim

To implement **Linear Regression** to predict the price of a house based on its area, number of bedrooms, and age.

Algorithm / Procedure

1. Create a house price dataset.
2. Select area, bedrooms, and age as input features.
3. Use house price as the target variable.
4. Train the Linear Regression model.
5. Provide details of a new house.
6. Predict and display its price.

Program



```
Predicted Car Price: 939583.33

[14] ✓ Os
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([
    [1000, 2, 10],
    [1500, 3, 5],
    [2000, 4, 2],
    [1200, 2, 8],
    [1800, 3, 4]
])

y = np.array([3000000, 4500000, 6500000, 3500000, 5500000])

model = LinearRegression()
model.fit(X, y)

house = [[1600, 3, 5]]
price = model.predict(house)

print("Predicted House Price:", round(price[0], 2))

... Predicted House Price: 4943181.82
```

Sample Output

Predicted House Price: 4800000.0

Result

Thus, the **House Price Prediction** model was successfully implemented using Linear Regression.

5. Iris Classification using Naïve Bayes

Aim

To implement the **Naïve Bayes classification algorithm** to classify Iris flowers into different species.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Create a Gaussian Naïve Bayes classifier.
4. Train the model using the training data.
5. Predict the species of the test data.
6. Calculate and display the accuracy.

Program

```
[15] ✓ 0s
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual   :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

... Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Actual   : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 2 1 1 0 1 2 1 2 ...]

Actual : [1 0 2 1 1 0 1 2 1 2 ...]

Accuracy : 1.0

Result

Thus, **Iris classification using the Naïve Bayes algorithm** was successfully implemented with high accuracy.

16. Comparison of Classification Algorithms

Aim

To implement and compare different **classification algorithms** based on their accuracy using the Iris dataset.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Apply KNN, Decision Tree, and Naïve Bayes classifiers.
4. Train each model using the training data.
5. Predict the test data.
6. Calculate the accuracy of each algorithm.
7. Compare the results.

Program



```
[16] ✓ Os
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

models = {
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB()
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, "Accuracy:", accuracy_score(y_test, pred))

*** KNN Accuracy: 1.0
    Decision Tree Accuracy: 1.0
    Naive Bayes Accuracy: 1.0
```

Sample Output

KNN Accuracy: 1.0

Decision Tree Accuracy: 1.0

Naive Bayes Accuracy: 1.0

Result

Thus, different **classification algorithms** were successfully implemented and their accuracies were compared using the Iris dataset.

17. Mobile Price Prediction

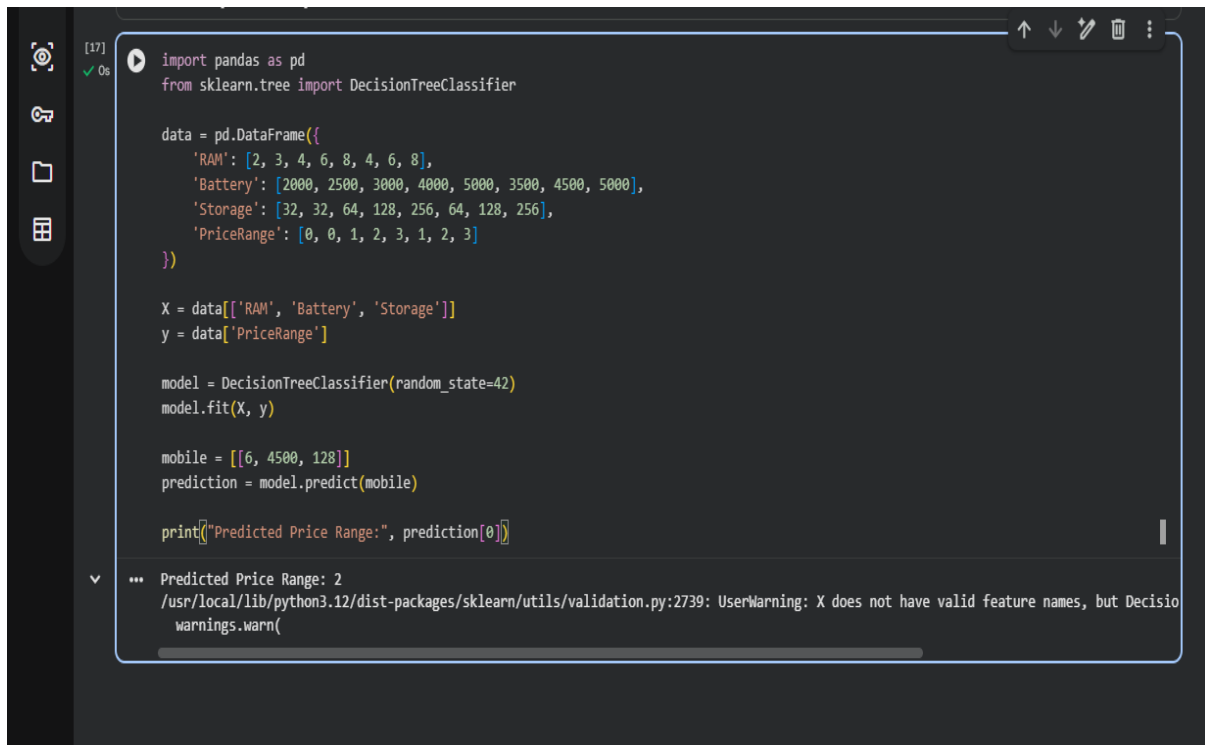
Aim

To implement a **classification model** to predict the price range of a mobile phone based on its features.

Algorithm / Procedure

1. Create a mobile phone dataset.
2. Select features such as RAM, battery, and storage.
3. Use price range as the target variable.
4. Split the dataset into training and testing data.
5. Train a Decision Tree classifier.
6. Predict the price range of a new mobile.
7. Display the prediction.

Program



```
[17] ✓ Os
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = pd.DataFrame({
    'RAM': [2, 3, 4, 6, 8, 4, 6, 8],
    'Battery': [2000, 2500, 3000, 4000, 5000, 3500, 4500, 5000],
    'Storage': [32, 32, 64, 128, 256, 64, 128, 256],
    'PriceRange': [0, 0, 1, 2, 3, 1, 2, 3]
})

X = data[['RAM', 'Battery', 'Storage']]
y = data['PriceRange']

model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

mobile = [[6, 4500, 128]]
prediction = model.predict(mobile)

print("Predicted Price Range:", prediction[0])

... Predicted Price Range: 2
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but DecisionTreeClassifier might require it.
warnings.warn(
```

Sample Output

Predicted Price Range: 2

Result

Thus, the **Mobile Price Prediction** model was successfully implemented using a Decision Tree classifier.

18. Perceptron-Based Iris Classification

Aim

To implement a **Perceptron algorithm** for classifying Iris flowers into different species.

Algorithm / Procedure

1. Load the Iris dataset.
2. Select two classes from the dataset.
3. Split the data into training and testing sets.
4. Create a Perceptron classifier.
5. Train the model using the training data.
6. Predict the test data.
7. Calculate and display the accuracy.

Program

```
[18] ✓ Os
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

data = load_iris()

X = data.data[:100]
y = data.target[:100]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = Perceptron(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual  :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

... Predicted: [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0]
Actual  : [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 1 0 0 1 1 0 1 0 ...]

Actual : [1 0 1 0 0 1 1 0 1 0 ...]

Accuracy : 1.0

Result

Thus, the **Perceptron algorithm** was successfully implemented for Iris classification with high accuracy.

19. Naïve Bayes Bank Loan Prediction

Aim

To implement the **Naïve Bayes algorithm** to predict whether a customer will be approved for a bank loan.

Algorithm / Procedure

1. Create a bank loan dataset.
2. Select income, age, and loan amount as input features.
3. Use loan approval as the target variable.
4. Train a Gaussian Naïve Bayes classifier.
5. Predict the loan approval for a new customer.
6. Display the result.

Program

```
[19]
import pandas as pd
from sklearn.naive_bayes import GaussianNB

data = pd.DataFrame({
    'Income': [25000, 40000, 50000, 70000, 30000, 80000, 60000, 35000],
    'Age': [25, 30, 35, 45, 28, 50, 40, 32],
    'LoanAmount': [200000, 300000, 250000, 400000,
                  200000, 500000, 350000, 250000],
    'Approved': [0, 1, 1, 1, 0, 1, 1, 0]
})

X = data[['Income', 'Age', 'LoanAmount']]
y = data['Approved']

model = GaussianNB()
model.fit(X, y)

customer = [[55000, 38, 300000]]
prediction = model.predict(customer)

print("Loan Approval:", "Approved" if prediction[0] == 1 else "Not Approved")

*** Loan Approval: Approved
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but GaussianNB
warnings.warn(
```

Sample Output

Loan Approval: Approved

Result

Thus, the **Naïve Bayes Bank Loan Prediction** model was successfully implemented to predict whether a customer will be approved for a loan.

20. Implement Future Sales Prediction using a Suitable Machine Learning Algorithm

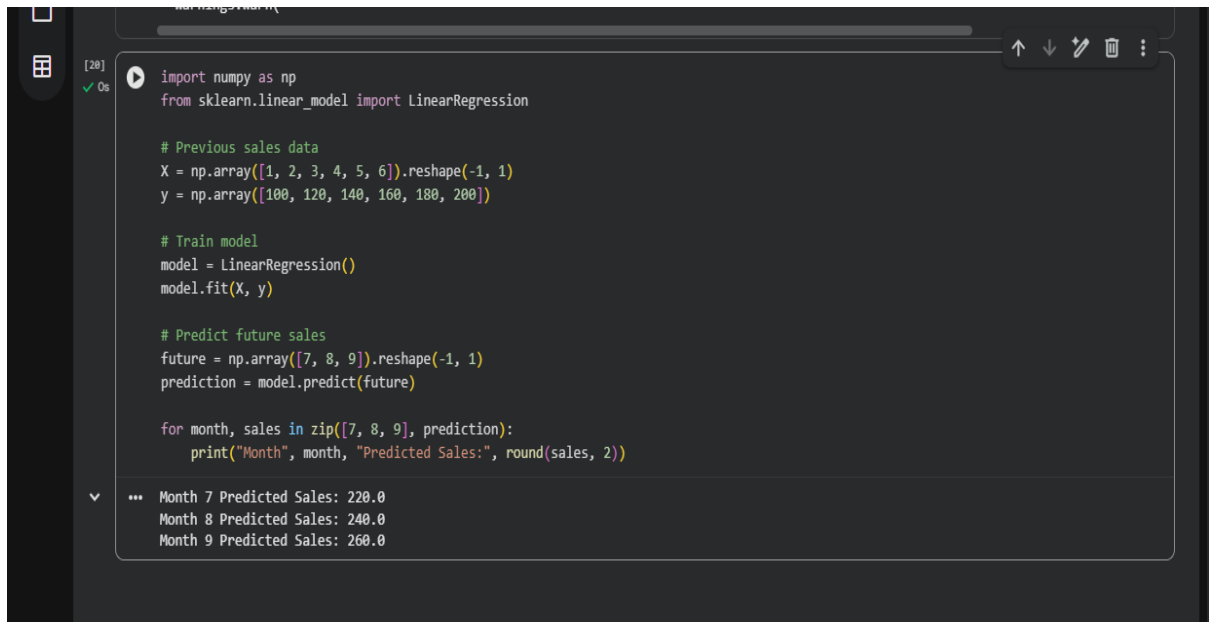
Aim

To implement **Future Sales Prediction** using the **Linear Regression** machine learning algorithm.

Algorithm / Procedure

1. Import the required libraries.
2. Create a dataset containing previous sales values.
3. Separate the input (Month) and output (Sales).
4. Train a **Linear Regression** model using the training data.
5. Predict sales for future months.
6. Display the predicted sales values.

Program



```
[20] import numpy as np
from sklearn.linear_model import LinearRegression

# Previous sales data
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([100, 120, 140, 160, 180, 200])

# Train model
model = LinearRegression()
model.fit(X, y)

# Predict future sales
future = np.array([7, 8, 9]).reshape(-1, 1)
prediction = model.predict(future)

for month, sales in zip([7, 8, 9], prediction):
    print("Month", month, "Predicted Sales:", round(sales, 2))
```

Month 7 Predicted Sales: 220.0
Month 8 Predicted Sales: 240.0
Month 9 Predicted Sales: 260.0

Sample Output

Month 7 Predicted Sales: 220.0

Month 8 Predicted Sales: 240.0

Month 9 Predicted Sales: 260.0

Result

Thus, future sales were successfully predicted using the Linear Regression machine learning algorithm.